1A. Implement Feedforward Neural Network (FNN) with Keras and TensorFlow for classification of hand-written MNIST dataset using below steps:

a. Import the necessary packages
b. Load the training and testing data
c. Define the network architecture using Keras
d. Train the model using SGD with 11 epochs
e. Evaluate the network
f. Plot the training loss and accuracy

1B. Implementing Feedforward Neural Network (FNN) with Keras and TensorFlow for the classification of Fashion MNIST dataset using below steps:

a. Import the necessary packages
b. Load the training and testing data (CIFAR10)
c. Define the network architecture using Keras
d. Train the model using SGD
e. Evaluate the network
f. Plot the training loss and accuracy

1C. Build Feedforward Neural Network (FNN) with Keras and TensorFlow for classification ofCIFAR10 image dataset using the following steps:

a. Import the necessary packages
b. Load the training and testing data
c. Define the network architecture using Keras
d. Train the model using SGD/Adam optimizer
e. Evaluate the network
f. Plot the training loss and accuracy

1E. Build Feedforward Neural Network (FNN) with Keras and TensorFlow for classification of Sign Language MNIST image dataset using the following steps:

a. Import the necessary packages
b. Load the training and testing data
c. Define the network architecture using Keras
d. Train the model using SGD/Adam optimizer
e. Evaluate the network
f. Plot the training loss and accuracy

2A. Implement the CNN model for classifying CIFAR10 image dataset by dividing themodel into following 4 stages:

a. Loading and preprocessing the image data
b. Defining the model's architecture
c. Training the model
d. Estimating the model's performance

2B. Implement the Image classification CNN model for classifying hand-written MNIST dataset by dividing the model into following 4 stages:

a. Loading and preprocessing the image data
b. Defining the model's architecture
c. Training the model
d. Estimating the model's performance

2C. Implement the CNN model for classifying Sign Language MNIST by dividing the model into following 4 stages:

a. Loading and preprocessing the image data
b. Defining the model's architecture
c. Training the model
d. Estimating the model's performance

3A. Implement credit card fraud detection for given credit card fraud detection Kaggle dataset using Autoencoder and build the model by using the following steps:

- **Dataset**: The dataset contains transactions labeled as fraudulent or non-fraudulent, with a severe class imbalance.
- **Goal:** Identify fraudulent transactions by training an autoencoder only on non-fraudulent transactions.

Steps:

- Preprocess the data by scaling features (features are already PCA-transformed, so they're anonymized).
- Train an autoencoder on non-fraudulent transactions to learn typical spending patterns.
- Calculate reconstruction errors for transactions and set a threshold to detect fraud.
- Compile the models with Optimizer, Loss, and Evaluation Metrics

3B. Implement Industrial Equipment Anomaly Detection on the MVTec AD Kaggle dataset using Autoencoder and build the model by using the following steps:

Steps:

- Preprocess images (resize, normalize) for input into a convolutional autoencoder.
- Build a convolutional autoencoder (CNN-based encoder-decoder) to learn the representations of normal object images.
- Use reconstruction errors in pixel values to detect anomalies in test images.

Dataset: MVTec AD is an anomaly detection dataset for industrial defect detection, containing images of normal and defective objects.

Goal: Detect defects in objects by training an image-based autoencoder on images of defect-free objects.

3C. Implement Anomaly Detection in Energy Consumption Data (Smart Grid Dataset) using Autoencoder and build the model by using the following steps:

Steps:

- Preprocess the data to fill any missing values, normalize, and create time windows for sequential input.
- Use an LSTM-based autoencoder to learn temporal patterns in normal consumption.
- Set an error threshold and detect anomalies when the reconstruction error exceeds this threshold.

Dataset: The Smart Grid dataset records electricity consumption across multiple timestamps for various households.

Goal: Detect unusual consumption patterns by training an autoencoder on typical (normal) energy usage patterns

3D. Detect the faults in Sensor Data from Machinery (NASA Turbofan Engine Dataset) using Autoencoder and build the model by using the following steps:

- **Dataset**: The dataset contains sensor readings from turbofan engines with labels indicating whether a reading is faulty.
- **Goal**: Detect machinery faults by training an autoencoder on sensor data from healthy engines.

Steps:

- Preprocess data by scaling sensor readings.
- Use an LSTM-based autoencoder (to capture sequential data) to learn patterns in normal sensor readings.
- Calculate reconstruction error for test data, with a threshold to classify high-error instances as faults.

3E. Use Autoencoder to implement Network Intrusion Detection on NSL-KDD dataset. Build the model

- **Dataset**: NSL-KDD is a network intrusion detection dataset with various types of network traffic, labeled as either "normal" or "attack."
- **Goal**: Identify network intrusions by training an autoencoder on normal network traffic data, then flagging traffic as anomalous if the reconstruction error is high.

Steps:

- Preprocess the NSL-KDD dataset, particularly by encoding categorical features and scaling numerical features.

- Define an autoencoder similar to the ECG example, focusing on the reconstruction of normal network patterns.
- Train on normal data and set an error threshold to detect intrusions.

3F. Using the Sign Language MNIST dataset, select any one of the 24 alphabet classes (label Y-24) as an anomaly. Train an Autoencoder–Decoder model using the remaining 23 classes and detect the selected class as an anomaly based on reconstruction error.

4A. Implement the CBOW for document classification using deep learning.

Steps (for Implementing CBOW for Document Classification):

1. **Data Preparation:** You need a labeled dataset for document classification. You can use datasets like the **IMDb movie reviews** dataset (for sentiment classification) or **20 Newsgroups** dataset (for topic classification).

   Load and Preprocess the data:

   - from tensorflow.keras.datasets import imdb # For loading
   - Pad sequences to ensure uniform input length
   - Convert the IMDB dataset to a list of sentences (each sentence is a list of word)
   - Tokenize the text into words.
   - Lowercase the text and remove stop words.
   - Create a vocabulary from the training data.

2. **Define CBOW Model:** CBOW learns to predict a target word based on surrounding context words. For document classification, you need to modify the CBOW model to work with document-level contexts. In CBOW, the context consists of words surrounding the target word, but for documents, you will use the embeddings from all the words in the document.

3. **Model Architecture:**
   - **Embedding Layer**: First, use the CBOW architecture to learn word embeddings from the text corpus.
   - **Document Representation**: After learning the word embeddings, represent a document by averaging the embeddings of all words in the document.
   - **Fully Connected Layer**: Use this document representation as input to a neural network (e.g., a dense layer) for classification.
   - Train word2vec model with CBOW (sg=0 for CBOW)
   - Convert words to embeddings
   - Build the feed forward neural network for document classification

4. **Evaluate the Model on test set using loss and accuracy.**

4B. Implement the Continuous Bag of Words (CBOW) Model for the twitter dataset (download from Kaggle) using the below steps:

a. **Data Collection and Preprocessing:** Use Kaggle Twitter dataset or any dataset with tweets. Tokenize the tweets, clean the text, and prepare it for training.
b. **Train Word2Vec CBOW Model**
c. **Evaluate Results**: After training, we'll check the learned word embeddings and possibly evaluate using some examples.

4C. Implement the Continuous Bag of Words (CBOW) Model for the given textual document3 using the below steps:

a. **Data preparation**
b. **Train Word2Vec CBOW Model**
c. **Evaluate Results:** After training, we'll check the learned word embeddings and possibly evaluate using some examples.

4D. Building a Recommendation System with CBOW Word Embeddings

Problem: Use CBOW word embeddings to build a content-based recommendation system. For example, recommend similar articles or tweets based on the embeddings of keywords in a user's input.

Steps:

1. **Data Collection and Preprocessing**: Gather a dataset (e.g., a collection of tweets, articles, or products) with textual content (descriptions, reviews, etc.). Tokenize the text data, clean the data, and prepare it for training.
2. **Train CBOW Model**:
    a. Train the Word2Vec model using CBOW to learn word embeddings from the text data or Train the Word2Vec CBOW model on a dataset of articles or tweets.
    b. For a given article (or tweet), calculate the average of the embeddings of all the words in the document.
    c. For each document in the dataset, calculate the cosine similarity between the document's average word embedding and the user's input embedding.
    d. **Document Representation**: Represent each document (e.g., tweet, product description, article) as a vector by averaging the embeddings of all words in the document.
3. **Recommendation Logic**: For a given input document (e.g., a user's search or a product), recommend the most similar documents using cosine similarity between their vector representations.

5A. Implement Object detection using Transfer Learning with CNN for the image

classification-caltec-101 dataset and VGG / ResNet pre-trained models.

Steps:

a. Set up the environment
b. Loading and preprocessing data
c. Choose the pre-trained Model (Transfer Learning)
d. Freeze the base layers and add custom layers for fine tuning
e. Object detection training
f. Compute loss
g. Evaluate the Model
h. Visualize the prediction
i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre-trained model and retraining it on the new dataset to adapt it better to your task.

5B. Object detection using Transfer Learning of CNN architectures for the Intel Image Classification Dataset using the below steps:

a. Set up the environment
b. Loading and preprocessing data
c. Choose the pre-trained Model (Transfer Learning)
d. Freeze the base layers and add custom layers for fine tuning
e. Object detection training
f. Compute loss
g. Evaluate the Model
h. Visualize the prediction
i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre-trained model and retraining it on the new dataset to adapt it better to your task.

5C. Object detection using Transfer Learning of CNN architectures for the Fashion MNIST dataset using the below steps:

a. Set up the environment
b. Loading and preprocessing data
c. Choose the pre-trained Model (Transfer Learning)
d. Freeze the base layers and add custom layers for fine tuning
e. Object detection training
f. Compute loss
g. Evaluate the Model
h. Visualize the prediction
i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre-trained model and retraining it on the new dataset to adapt it better to your task.